

SotilityBadgeNFT.sol

SotilityBadgeNFT.sol

Purpose:

Mint soulbound NFTs for:

Sotilitarian Status milestones

Utilitier Level achievements (e.g., seasonal impact)

DAO or AI oracle can issue badges

Fully non-transferable — proof of contribution

A badge system that is soulbound, tiered, and IPFS-powered.

Key Traits:

ERC721-compliant

Mint controlled by MINTER_ROLE

Metadata served from IPFS

Optional override for tiers/badge types

Example Use:

Mint IPFS metadata for badges like:

sotility.com/badges/impact-season-1.json

sotility.com/badges/status-platinum.json

Link it visually in SoGoodDashboard

--

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";

/**
 * @title SotilityBadgeNFT
 * @dev Soulbound NFT badges for status, impact, and tier recognition.
 */
contract SotilityBadgeNFT is ERC721URIStorage, AccessControl {
    bytes32 public
    constant MINTER_ROLE =
    keccak256("MINTER_ROLE");

    uint256 private _tokenIds;
    mapping(uint256 => bool) public
    soulbound;

    event BadgeMinted(address indexed
    to, uint256 tokenId, string uri);
    event BadgeRevoked(uint256
    tokenId);

    constructor() ERC721("Sotility Badge
    NFT", "SBNFT") {
        _grantRole(DEFAULT_ADMIN_ROLE,
        msg.sender);
    }

    function mint(address to, string
    calldata uri, bool isSoulbound) external
    onlyRole(MINTER_ROLE) returns
    (uint256) {
        _tokenIds++;
        uint256 newId = _tokenIds;
        _mint(to, newId);
        _setTokenURI(newId, uri);
        soulbound[newId] = isSoulbound;
        emit BadgeMinted(to, newId, uri);
        return newId;
    }

    function revoke(uint256 tokenId)
    external onlyRole(MINTER_ROLE) {
        _burn(tokenId);
        emit BadgeRevoked(tokenId);
    }

    // Override to disable soulbound
    transfer
    function _beforeTokenTransfer(
        address from,
        address to,
        uint256 tokenId,
        uint256 batchSize
    ) internal override {
        super._beforeTokenTransfer(from,
        to, tokenId, batchSize);
        require(from == address(0) || !
        soulbound[tokenId], "Soulbound:
        Non-transferable");
    }

    function supportsInterface(bytes4
    interfacedId)
        public view override(ERC721,
        AccessControl)
        returns (bool)
    {
        return
        super.supportsInterface(interfacedId);
    }
}
```

